

Grafì di precedenza e parallelismo

Alessia Smeralda Brunello

Matricola: 544964

Hands-On 10

Indice

1	Introduzione	2
2	Definizione del problema	2
3	Metodologia	2
3.1	Grafo di precedenza	2
3.2	Implementazione in C	3
3.3	Implementazione in Python	3
4	Presentazione dei risultati	4
4.1	Compilazione	4
4.2	Output del programma C	4
4.3	Output del programma Python	5
4.4	Analisi	5
5	Estensione ai processi	6
6	Conclusioni	7

1 Introduzione

Il parallelismo rappresenta un aspetto fondamentale nei sistemi di calcolo moderni, in quanto consente di migliorare le prestazioni attraverso l'esecuzione concorrente di più operazioni.

In questo contesto, l'obiettivo dell'esercizio è analizzare il parallelismo implicito in un'espressione matematica mediante la costruzione di un grafo di precedenza e la sua successiva implementazione tramite thread.

In particolare, si vuole valutare l'espressione:

$$y = (2 * 6) + (1 + 4) * (5 - 2)$$

utilizzando sia il linguaggio C (con thread POSIX) sia Python (con il modulo threading), evidenziando le differenze tra i due approcci.

2 Definizione del problema

Il problema consiste nel decomporre l'espressione in operazioni elementari e individuare le dipendenze tra di esse.

Operazioni:

- $A = 2 * 6$
- $B = 1 + 4$
- $C = 5 - 2$
- $D = B * C$
- $Y = A + D$

Dipendenze:

- A, B, C indipendenti
- D dipende da B e C
- Y dipende da A e D

Questa struttura definisce un grafo di precedenza.

3 Metodologia

3.1 Grafo di precedenza

Il grafo di precedenza associato all'espressione

$$Y = (2 * 6) + (1 + 4) * (5 - 2)$$

è riportato in Figura 1. Le operazioni A , B e C sono indipendenti e possono quindi essere eseguite in parallelo; il nodo D dipende dai risultati di B e C , mentre il nodo finale Y dipende da A e D .

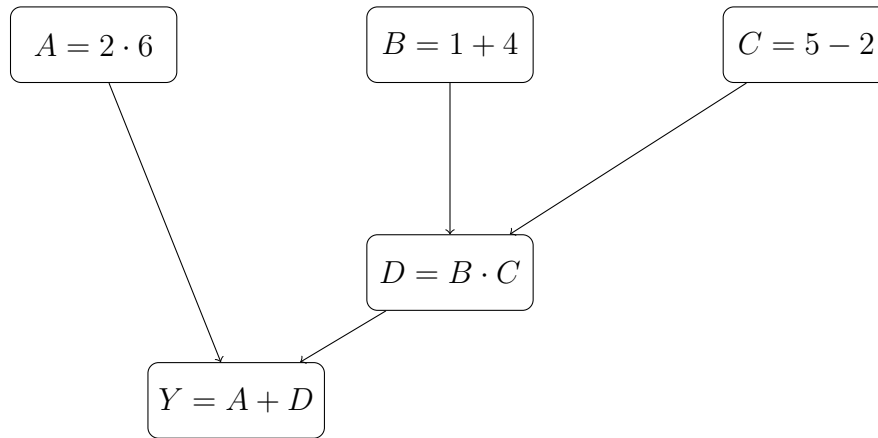


Figura 1: Grafo di precedenza dell'espressione assegnata.

3.2 Implementazione in C

Il programma è stato realizzato in modo modulare, suddividendo il codice in più file.

Ogni operazione è stata implementata come funzione eseguita da un thread. I valori sono restituiti tramite memoria dinamica (`malloc`) per garantirne la validità oltre la durata del thread.

Per le operazioni che richiedono più input, sono state utilizzate strutture dati (`struct`) per passare i parametri ai thread.

La sincronizzazione è stata gestita mediante `pthread_join`, in modo da garantire il rispetto delle dipendenze definite dal grafo di precedenza.

In particolare, le operazioni indipendenti sono state eseguite in parallelo, mentre le operazioni dipendenti sono state avviate solo dopo il completamento dei thread necessari.

3.3 Implementazione in Python

In Python è stato utilizzato il modulo `threading`.

A differenza del C, non è necessario gestire manualmente la memoria né utilizzare puntatori. I risultati sono stati condivisi tra i thread mediante un dizionario globale.

La sincronizzazione è stata ottenuta con il metodo `join()`, garantendo il corretto ordine di esecuzione.

4 Presentazione dei risultati

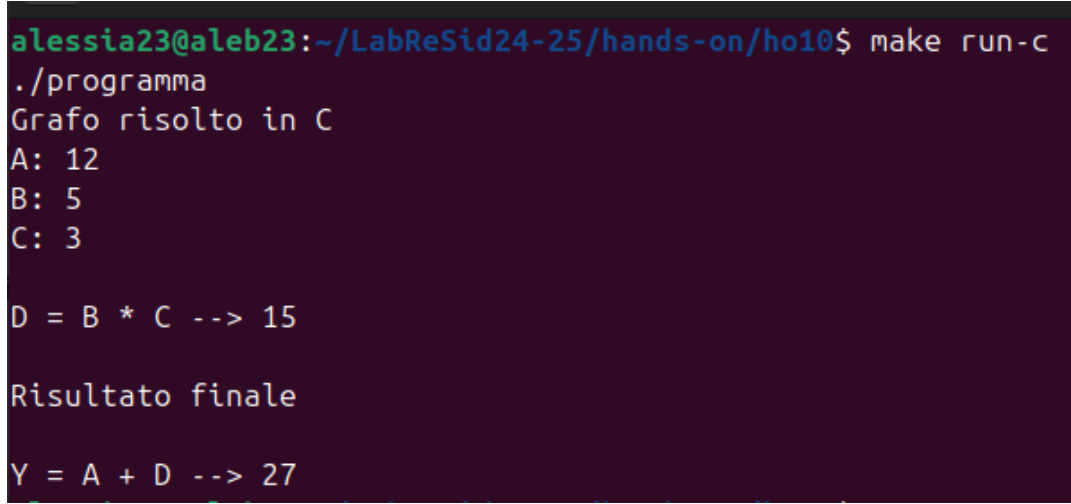
4.1 Compilazione

La compilazione è stata gestita tramite `Makefile`. Il progetto produce un eseguibile: **programma**

I principali target usati per l'esecuzione sono:

- `run-c`, per eseguire il primo esercizio;
- `run-python`, per eseguire il secondo esercizio.

4.2 Output del programma C



```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho10$ make run-c
./programma
Grafo risolto in C
A: 12
B: 5
C: 3

D = B * C --> 15

Risultato finale

Y = A + D --> 27
```

4.3 Output del programma Python

```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho10$ make run-python
python3 ho10.py
Grafo risolto in Python

A: 12

B: 5

C: 3

D = B * C --> 15

Risultato

Y = A + D --> 27
```

I risultati ottenuti confermano la corretta esecuzione del programma e il rispetto delle dipendenze tra le operazioni definite dal grafo di precedenza.

4.4 Analisi

Linguaggio C:

- è necessaria una gestione esplicita della memoria tramite allocazione dinamica
- il passaggio dei dati tra thread avviene mediante puntatori
- è richiesto un maggiore controllo sui dettagli implementativi

Python:

- la gestione della memoria è automatica
- la condivisione dei dati tra thread è più semplice
- l'implementazione risulta più compatta e leggibile

Questo confronto evidenzia come Python offra un approccio più immediato, mentre il C garantisce un controllo più fine sulle risorse e sul comportamento dei thread.

5 Estensione ai processi

La traccia richiede inoltre di discutere se lo stesso schema possa essere realizzato anche tramite processi. La risposta è affermativa: il grafo di precedenza può essere implementato non solo con thread, ma anche con processi distinti.

Nel caso dei thread, tutti i flussi di esecuzione appartengono allo stesso processo e condividono lo stesso spazio di memoria. Questo rende più semplice lo scambio dei dati intermedi, poiché i risultati delle operazioni possono essere salvati in variabili o strutture condivise.

Nel caso dei processi, invece, ogni processo possiede uno spazio di memoria separato. Di conseguenza, anche se è possibile creare più processi per eseguire i nodi del grafo, i risultati parziali non possono essere condivisi in modo diretto come avviene con i thread. Per questo motivo è necessario utilizzare meccanismi di comunicazione o memoria condivisa.

In linguaggio C, la creazione dei processi può essere effettuata mediante la funzione `fork()`, mentre la sincronizzazione tra processo padre e processi figli può essere gestita con `wait()` oppure `waitpid()`.

Per condividere i risultati intermedi si possono utilizzare:

- memoria condivisa, ad esempio con `mmap()`
- System V shared memory, tramite `shmget()` e `shmat()`
- pipe, se si vogliono trasmettere i risultati da un processo all'altro

In Python è possibile seguire la stessa logica utilizzando il modulo `multiprocessing`. In questo caso i processi possono essere creati con `Process`, mentre la condivisione dei dati può essere realizzata attraverso strumenti come:

- Value
- Array
- Queue
- Pipe

Dal punto di vista concettuale, il grafo di precedenza non cambia: le operazioni A , B e C possono essere eseguite in parallelo, l'operazione D deve attendere il completamento di B e C , e infine l'operazione Y può essere calcolata solo dopo che sono disponibili A e D . Cambia però il meccanismo con cui i risultati vengono scambiati e sincronizzati.

Rispetto ai thread, l'uso dei processi rende l'implementazione più complessa, perché richiede strumenti aggiuntivi per la comunicazione tra le unità di esecuzione. Tuttavia, dal punto di vista teorico, rappresenta comunque una soluzione valida al problema.

6 Conclusioni

L'esercizio ha permesso di comprendere come un problema apparentemente sequenziale possa essere trasformato in un problema parallelo attraverso l'utilizzo di grafi di precedenza.

L'implementazione in C ha evidenziato la complessità legata alla gestione esplicita della memoria e del passaggio dei dati tra thread, mentre Python ha mostrato un approccio più semplice e immediato grazie alla gestione automatica delle risorse.

Dal punto di vista concettuale, il grafo di precedenza rappresenta lo strumento fondamentale per individuare le opportunità di parallelismo e per progettare correttamente l'esecuzione concorrente.

Infine, è stato osservato come lo stesso schema possa essere implementato anche mediante processi, sebbene con maggiore complessità dovuta alla necessità di meccanismi di comunicazione tra processi.

L'esercizio evidenzia quindi l'importanza della fase di progettazione del parallelismo prima dell'implementazione, indipendentemente dalla tecnologia utilizzata.